Technical Specifications: Smart Traffic Monitoring Application

Version: 1.0
Date: February 6, 2026
Target Audience: Development Team, Computer Vision Engineers

---

1. Executive Summary

This document specifies the requirements for a real-time, multi-lane Smart Traffic Monitoring System using computer vision and deep learning. The system will detect, track, and analyze vehicle traffic with automatic direction calibration, wrong-way detection, and intelligent traffic signal control.

Primary Use Cases:

- Final-year engineering projects
- Smart city traffic demonstrations
- Traffic analytics research
- Intelligent Transportation System (ITS) prototypes

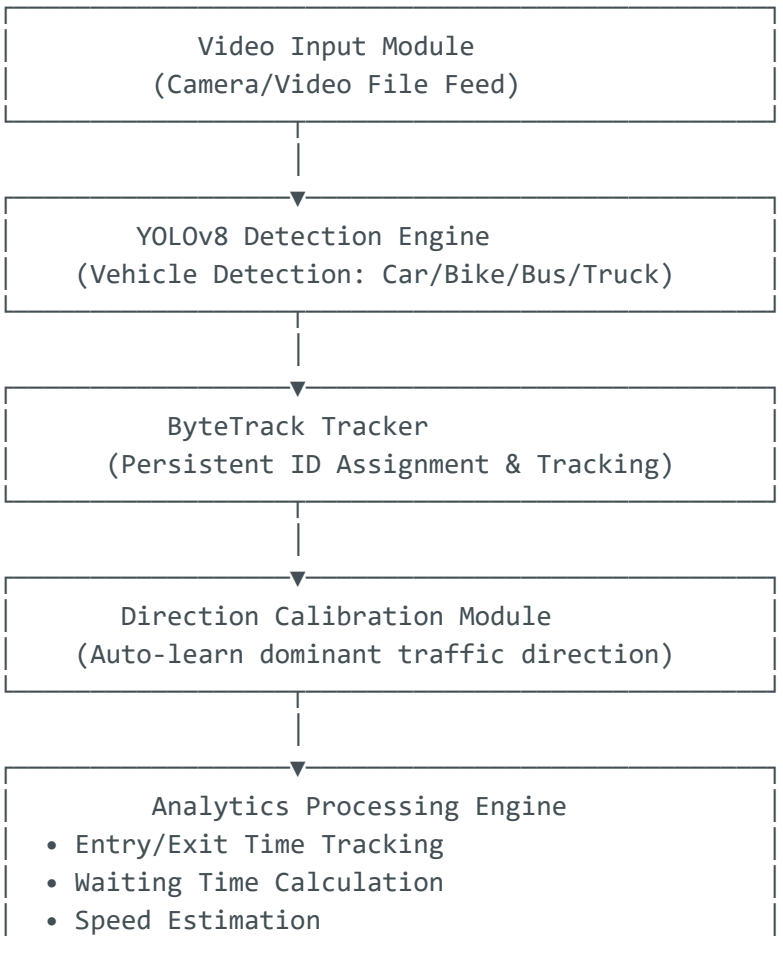
---

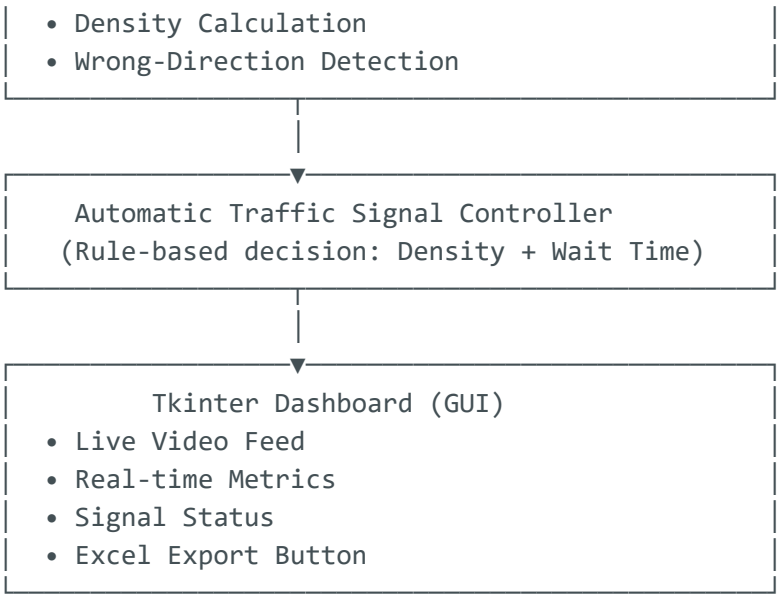
2. System Architecture

2.1 Technology Stack

Component	Technology	Version/Notes
Programming Language	Python	3.8+
Object Detection	YOLOv8	YOLOv8n (nano) for real-time performance
Object Tracking	ByteTrack	Multi-object tracking with persistent IDs
Computer Vision	OpenCV	cv2 library
GUI Framework	Tkinter	Desktop interface
Data Export	OpenPyXL	Excel file generation
Numerical Processing	NumPy	Array operations
Image Processing	PIL/Pillow	ImageTk for Tkinter integration

2.2 System Components





---

3. Functional Requirements

3.1 Vehicle Detection

FR-1.1: The system SHALL detect only the following vehicle classes:

- Car
- Motorcycle
- Bus
- Truck

FR-1.2: The system SHALL use YOLOv8n model for real-time detection performance.

FR-1.3: Detection confidence threshold SHALL be configurable (recommended: 0.3-0.5).

FR-1.4: The system SHALL filter out non-vehicle objects (pedestrians, animals, etc.).

3.2 Vehicle Tracking

FR-2.1: The system SHALL use ByteTrack for persistent object ID assignment.

FR-2.2: The system SHALL prevent double-counting of the same vehicle.

FR-2.3: The system SHALL handle vehicle occlusions and maintain ID consistency.

FR-2.4: The system SHALL support vehicle re-identification after temporary disappearance.

FR-2.5: Track IDs SHALL persist throughout the vehicle's presence in the frame.

3.3 Multi-Lane Support

FR-3.1: The system SHALL process vehicles across multiple lanes simultaneously.

FR-3.2: The system SHALL NOT use hardcoded lane boundaries.

FR-3.3: Vehicle processing SHALL be based on motion direction, not spatial position.

FR-3.4: The system SHALL handle varying lane configurations without reconfiguration.

3.4 Automatic Direction Calibration

FR-4.1: The system SHALL automatically learn the dominant traffic direction during initialization.

FR-4.2: Direction calibration SHALL NOT assume any fixed movement pattern (top-to-bottom, etc.).

FR-4.3: The system SHALL calculate average motion vectors over initial calibration frames (recommended: 50-100 frames).

FR-4.4: The system SHALL support rotated, tilted, or inverted camera placements.

FR-4.5: Dominant direction SHALL be stored as a normalized vector.

3.5 Wrong-Direction Vehicle Detection

FR-5.1: The system SHALL detect vehicles moving opposite to the learned dominant direction.

FR-5.2: Wrong-direction vehicles SHALL be:

- Highlighted in RED on the video feed
- Excluded from traffic analytics (speed, density)
- Counted separately as violations

FR-5.3: The system SHALL maintain a dedicated counter for wrong-direction vehicles.

FR-5.4: Wrong-direction threshold SHALL be configurable (recommended: angle > 120° from dominant direction).

3.6 Entry & Exit Time Tracking

FR-6.1: The system SHALL record entry timestamp when a vehicle is first detected.

FR-6.2: The system SHALL record exit timestamp when a vehicle leaves the frame or tracking is lost.

FR-6.3: Timestamps SHALL be stored in `YYYY-MM-DD HH:MM:SS` format.

FR-6.4: The system SHALL maintain per-vehicle time logs indexed by unique track ID.

3.7 Waiting Time Calculation

FR-7.1: The system SHALL calculate waiting time automatically for each vehicle.

FR-7.2: A vehicle SHALL be considered "waiting" when:

- Movement is below pixel threshold (recommended: 2-5 pixels per frame)

FR-7.3: Waiting time SHALL be:

- Accumulated continuously while vehicle is stationary
- Stored per vehicle ID
- Included in analytics and export

FR-7.4: Total waiting time SHALL be calculated as sum of all waiting intervals.

3.8 Automatic Traffic Signal Logic

FR-8.1: The system SHALL implement fully automatic traffic signal control.

FR-8.2: Manual toggle controls for traffic signals are PROHIBITED.

FR-8.3: Signal state SHALL be determined by:

- Current vehicle density

- Average waiting time across all vehicles

FR-8.4: Signal decision rules:

- **RED:** High density (>threshold) OR high average waiting time (>threshold)
- **GREEN:** Low density AND low average waiting time
- Recommended thresholds: Density > 10 vehicles, Waiting time > 30 seconds

FR-8.5: Signal state SHALL update continuously in real-time.

FR-8.6: Signal transitions SHALL be logged with timestamps.

3.9 Speed & Density Analytics

FR-9.1: The system SHALL estimate per-vehicle speed using pixel displacement.

FR-9.2: Speed calculation formula:

```
Speed (pixels/frame) = Distance moved / Time elapsed
```

FR-9.3: The system SHALL calculate and display:

- Average speed across all active vehicles
- Current traffic density (number of vehicles in frame)

FR-9.4: Metrics SHALL update in real-time at minimum 1 Hz.

FR-9.5: Speed estimation SHALL exclude stationary or slow-moving vehicles (waiting).

3.10 Real-Time Dashboard (GUI)

FR-10.1: The system SHALL implement a Tkinter-based desktop GUI.

FR-10.2: Minimum window dimensions: 1400×800 pixels.

FR-10.3: The dashboard SHALL display:

- Live video feed with detection overlays
- Total vehicle count (valid direction only)
- Average speed (pixels/frame or km/h if calibrated)

- Current density (vehicles in frame)
- Wrong-direction violation count
- Current traffic signal state (RED/GREEN indicator)

FR-10.4: The GUI SHALL be responsive and update at minimum 15 FPS.

FR-10.5: UI elements SHALL be clearly labeled and professionally styled.

FR-10.6: The system SHALL handle window resize events gracefully.

3.11 Excel Export Functionality

FR-11.1: The system SHALL provide a "Download Excel" button in the GUI.

FR-11.2: Excel export SHALL generate one row per tracked vehicle.

FR-11.3: Required columns:

- Vehicle ID (Track ID)
- Vehicle Type (Car/Motorcycle/Bus/Truck)
- Entry Time (timestamp)
- Exit Time (timestamp)
- Total Waiting Time (seconds)

FR-11.4: Excel file SHALL be saved with timestamp in filename (e.g., `traffic_data_2026-02-06_14-30-45.xlsx`).

FR-11.5: Export SHALL only include vehicles that have exited (complete data).

FR-11.6: File SHALL be analysis-ready with proper headers and formatting.

---

4. Non-Functional Requirements

4.1 Performance Requirements

NFR-1.1: The system SHALL operate in real-time on CPU-only systems.

NFR-1.2: Frame processing rate SHALL be minimum 10 FPS on mid-range hardware.

NFR-1.3: The system SHALL implement frame skipping for performance optimization (recommended: process every 2-3 frames).

NFR-1.4: Memory usage SHALL remain stable during extended operation (no memory leaks).

NFR-1.5: The system SHALL handle video feeds up to 1920×1080 resolution.

NFR-1.6: Tracking overhead SHALL not exceed 100ms per frame.

4.2 Reliability Requirements

NFR-2.1: The system SHALL handle missing detections gracefully without crashes.

NFR-2.2: Track IDs SHALL be managed safely to prevent overflow or collision.

NFR-2.3: The system SHALL recover from temporary video feed interruptions.

NFR-2.4: All data structures SHALL be thread-safe if multi-threading is implemented.

4.3 Code Quality Requirements

NFR-3.1: Code SHALL follow modular structure with clear separation of concerns.

NFR-3.2: Functions and variables SHALL use descriptive, self-documenting names.

NFR-3.3: Redundant logic and code duplication SHALL be eliminated.

NFR-3.4: All features SHALL work together without conflicts.

NFR-3.5: The application SHALL be runnable without missing dependencies.

NFR-3.6: Code SHALL include inline comments for complex algorithms.

NFR-3.7: Error handling SHALL be implemented for all critical operations.

4.4 Usability Requirements

NFR-4.1: The system SHALL be operable by non-technical users for basic functions.

NFR-4.2: Setup and initialization SHALL require minimal configuration.

NFR-4.3: Error messages SHALL be clear and actionable.

---

5. System Constraints

5.1 Hardware Constraints

C-1.1: Minimum CPU: Intel i5 or equivalent (4 cores) **C-1.2:** Minimum RAM: 8 GB **C-1.3:** GPU: Optional (CUDA-enabled for enhanced performance) **C-1.4:** Camera: USB webcam or IP camera with RTSP support

5.2 Software Constraints

C-2.1: Operating System: Windows 10/11, Linux (Ubuntu 20.04+), macOS 11+ **C-2.2:** Python Version: 3.8 or higher **C-2.3:** Dependencies must be installable via pip

5.3 Environmental Constraints

C-3.1: The system SHALL operate in varying lighting conditions (day/night). **C-3.2:** The system SHALL handle weather variations (rain, fog - reduced accuracy acceptable). **C-3.3:** Camera SHALL have unobstructed view of traffic lanes.

---

6. Data Specifications

6.1 Vehicle Data Structure

```
{
    "track_id": int,              # Unique tracking ID
    "vehicle_type": str,          # "Car", "Motorcycle", "Bus", "Truck"
    "entry_time": datetime,       # First detection timestamp
    "exit_time": datetime,        # Last detection timestamp
    "total_waiting_time": float,  # Seconds
    "positions": List[Tuple],     # [(x, y, frame_num), ...]
    "is_wrong_direction": bool    # True if moving opposite to flow
}
```

6.2 Analytics Data Structure

```
{
    "total_vehicles": int,           # Count of valid-direction vehicles
    "wrong_direction_count": int,    # Violation count
    "current_density": int,          # Vehicles currently in frame
    "average_speed": float,          # Pixels per frame
    "average_waiting_time": float,   # Seconds
    "signal_state": str              # "RED" or "GREEN"
}
```

6.3 Configuration Parameters

Parameter	Type	Default	Description
`detection_confidence`	float	0.4	YOLOv8 confidence threshold
`frame_skip`	int	2	Process every Nth frame
`calibration_frames`	int	100	Frames for direction learning
`waiting_threshold`	float	3.0	Pixels/frame below = waiting
`wrong_dir_angle`	float	120.0	Angle threshold (degrees)
`density_red_threshold`	int	10	Vehicles for RED signal
`wait_time_red_threshold`	float	30.0	Seconds for RED signal

7. Algorithm Specifications

7.1 Direction Calibration Algorithm

```
ALGORITHM: AutoDirectionCalibration
INPUT: video_frames[1..N], N = calibration_frames
OUTPUT: dominant_direction_vector

1. motion_vectors = []
2. FOR each frame in calibration_frames:
3.     detections = YOLOv8.detect(frame)
```

```
    tracks = ByteTrack.update(detections)
    FOR each track WITH history > 5 frames:
        current_pos = track.current_position
        prev_pos = track.previous_position
        motion = (current_pos - prev_pos)
        IF magnitude(motion) > threshold:
            motion_vectors.append(normalize(motion))
dominant_direction = mean(motion_vectors)
dominant_direction = normalize(dominant_direction)
RETURN dominant_direction
```

7.2 Waiting Time Calculation Algorithm

```
ALGORITHM: CalculateWaitingTime
INPUT: track_object, current_frame
OUTPUT: updated_waiting_time

current_pos = track.current_position
prev_pos = track.previous_position
displacement = magnitude(current_pos - prev_pos)
IF displacement < waiting_threshold:
    track.waiting_time += frame_interval
RETURN track.waiting_time
```

7.3 Traffic Signal Decision Algorithm

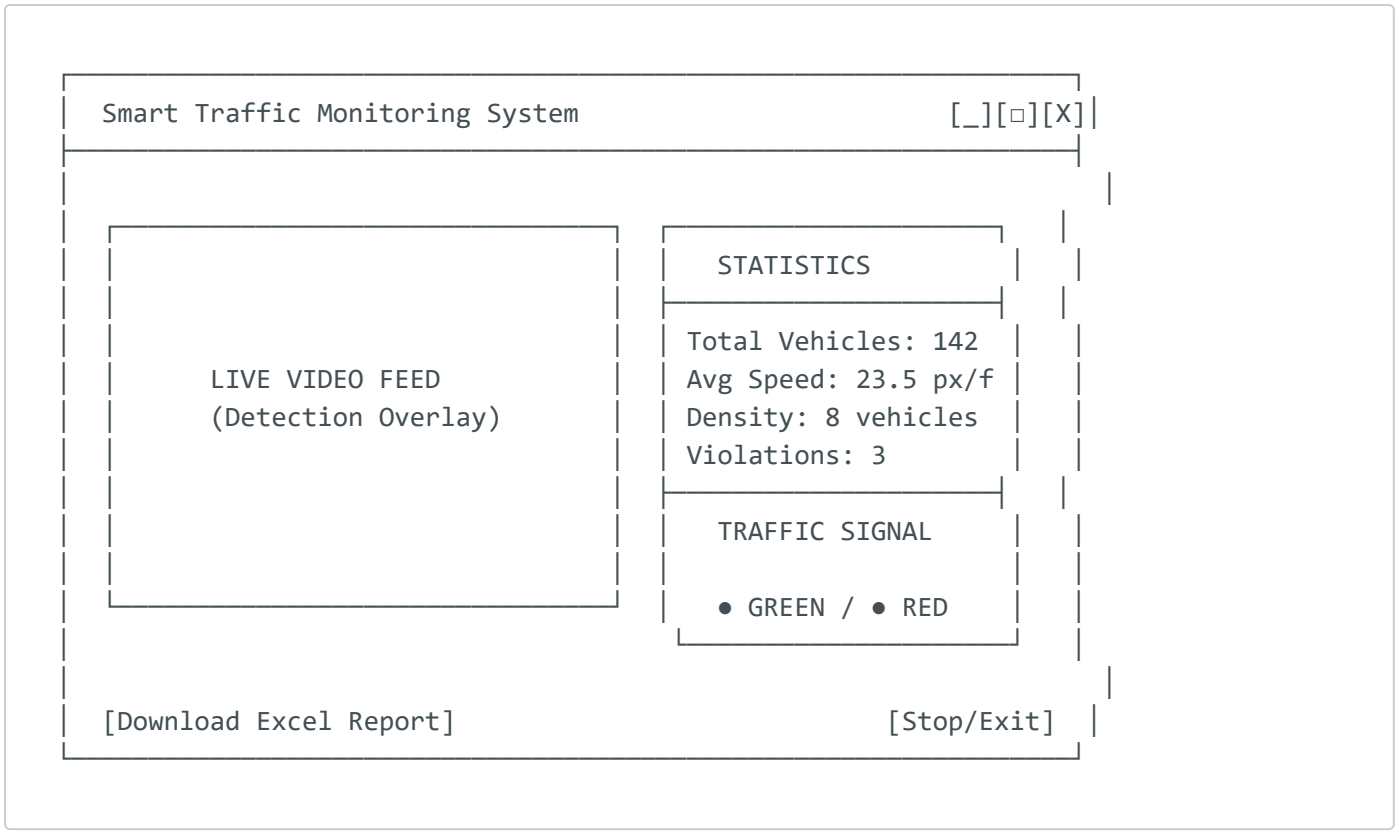
```
ALGORITHM: DetermineSignalState
INPUT: current_density, average_waiting_time
OUTPUT: signal_state ("RED" or "GREEN")

IF current_density > density_threshold:
    RETURN "RED"
IF average_waiting_time > wait_time_threshold:
    RETURN "RED"
RETURN "GREEN"
```

---

8. User Interface Specifications

8.1 Main Window Layout



8.2 Visual Indicators

- **Valid vehicles:** Green bounding box + Track ID label
- **Wrong-direction vehicles:** Red bounding box + "VIOLATION" label
- **Waiting vehicles:** Yellow bounding box (optional enhancement)
- **Traffic signal:** Circular indicator (GREEN/RED)

---

9. Testing Requirements

9.1 Unit Testing

- Vehicle detection accuracy (>85% precision)
- Tracking ID persistence (>90% consistency)
- Direction calibration accuracy (>95% correct direction)
- Waiting time calculation accuracy (±5% error margin)

9.2 Integration Testing

- End-to-end vehicle lifecycle (detection → tracking → exit → export)

- Signal state transitions based on varying traffic conditions
- GUI responsiveness under load

9.3 Performance Testing

- Frame processing rate under different video resolutions
- Memory usage over 60-minute continuous operation
- CPU utilization (should remain <80% on target hardware)

---

10. Deployment Requirements

10.1 Installation Package

The system SHALL be delivered with:

1. Python source code (modular structure)
2. `requirements.txt` with all dependencies
3. Pre-trained YOLOv8n model weights
4. Sample video files for testing
5. Configuration file (JSON/YAML)
6. README with setup instructions

10.2 Dependencies List

```
ultralytics>=8.0.0
opencv-python>=4.8.0
numpy>=1.24.0
Pillow>=10.0.0
openpyxl>=3.1.0
# ByteTrack (include installation instructions)
```

---

11. Optional Enhancements

These features are NOT required but may be implemented for additional value:

OPT-1: Camera calibration for real-world speed conversion (km/h) **OPT-2:** Lane-wise analytics (if lanes can be defined) **OPT-3:** Cloud database integration (Firebase, MongoDB) **OPT-4:** Violation snapshot capture (save images of wrong-direction vehicles) **OPT-5:** Adaptive signal optimization using reinforcement learning **OPT-6:** Heatmap visualization of traffic density **OPT-7:** Historical data analysis dashboard **OPT-8:** Email/SMS alerts for violations

---

12. Success Criteria

The project SHALL be considered complete when:

1. All functional requirements (FR-1 through FR-11) are implemented
2. Non-functional requirements (NFR-1 through NFR-4) are met
3. The system operates stably for 60+ minutes without crashes
4. Excel export generates valid, analysis-ready data
5. Direction calibration works correctly in 4+ different camera orientations
6. Wrong-direction detection achieves >90% accuracy
7. The application can be demonstrated on standard laptop hardware

---

13. Glossary

- **ByteTrack:** Multi-object tracking algorithm that assigns persistent IDs
- **Dominant Direction:** The primary flow direction of traffic learned during calibration
- **Track ID:** Unique identifier assigned to each vehicle throughout its presence
- **Waiting Time:** Accumulated time when vehicle movement is below threshold
- **Density:** Number of vehicles currently visible in the frame
- **Violation:** Vehicle moving opposite to dominant traffic direction

---

14. Document Control

Version	Date	Author	Changes
1.0	2026-02-06	Technical Team	Initial specification

Approval:

- ☐ Project Manager
- ☐ Lead Developer
- ☐ Quality Assurance

END OF TECHNICAL SPECIFICATIONS